SNOBI

THE PROJECT MASTER PLAN

Brand concept • Phase 0 website • UI/UX scope • Image direction • Codebase status • Build roadmap

A practical handover document for continuing the project outside Lovable and turning the current website prototype into a working clothing brand storefront.

Current checkpoint: Lovable generated the Phase 0 website through the Privacy Policy / Terms stage. The code has been pushed to GitHub, and the next step is to run, inspect, test, and then build the missing real commerce functionality.

Repository

github.com/Seif-shawky/welcome-home

Prepared from the project discussion and the publicly visible repository information checked on 28 October 2026. Implementation status should be re-verified locally before treating any checklist item as complete.

1. Executive summary

Snobi is a clothing brand built around a deliberately silly, broke, ambitious character. Customers are invited to “help” Snobi, and a T-shirt comes with the purchase. The website should feel like following a character’s ridiculous progress—not like a polished charity donation portal.

The first version should be intentionally simple and sketch-like. The story and scene can evolve over time, while the clothing designs themselves should still feel considered and worth buying.

The current codebase is best treated as a frontend-first prototype rather than a production-ready e-commerce application. The public repository shows a Lovable-created project and a roadmap that calls out unfinished real-product, Shopify, cart, checkout, supporter, and contact integrations. The stack can support server-side code, but the actual implemented backend and configuration must be inspected in the local codebase.

What success looks like

- A visitor understands Snobi’s joke and the “purchase = helping Snobi + receiving a shirt” concept.
- The visitor can browse real products, choose a valid size/variant, add it to a cart, pay through a supported provider, and receive an accurate confirmation.
- The story changes through deliberate phases, without spoiling future milestones.
- The website never pretends a mock product, payment, order, supporter, or donation is real when it is not.
- The project remains maintainable as a portfolio-quality coding project, with tests, version control, and documented decisions.

2. Brand concept and creative rules

Core premise

- **Character:** Snobi is broke, a bit dumb, ambitious, and confidently unrealistic.
- **Mechanic:** people “donate” or help Snobi by buying a shirt; be transparent that the shirt is included with the purchase.
- **Story:** each phase shows what Snobi has, what he spent money on, and what ridiculous goal he is chasing next.
- **Limited drops:** each phase can have a small, curated set of designs; items may disappear when a phase ends.
- **Community:** a “Who helped Snobi?” section could recognize supporters, but only with real verified purchases and appropriate consent.

Tone and visual direction

- Crude, hand-drawn, marker-like illustration; slightly awkward rather than slick or corporate.
- A simple, sketchbook / paper feeling with restrained colors and generous empty space.
- Humour should feel dumb and direct—not motivational, over-written, or cringe.
- The website should not over-design the story. One strong character scene and clear buying actions are better than many competing elements.
- Do not use fake urgency, fake supporter rankings, fake order counts, or unverified financial claims.

Story elements discussed

- Snobi starts on the ground beside an Egyptian roadside, with nothing and an empty wallet.
- A visible milestone or story history can explain what changed between phases.

- Potential future additions: “Who helped Snobi?”, “What did Snobi spend it on?”, a girlfriend character, and eventually an enemy.
- An Instagram account can become part of the story and tease the next phase.
- Milestones should become increasingly ridiculous and need not be only about money (for example, moving from a phone to a car to a house).

Naming and copy note: “Snobi one day or day one” was considered too cringe. Keep early phrases short, dumb, and character-led; final copy is still an open creative decision.

3. Phase 0 website scope

Phase 0 is the launch-ready starting point: Snobi has nothing, the first limited drop introduces the premise, and the site needs enough structure to support browsing and a real purchase. The UI/UX prompt discussed the following page set.

Page / area	Purpose and expected behavior
1. Home	Introduce “Snobi is broke,” show Snobi in the roadside scene, explain that buying a shirt helps Snobi and includes a shirt, feature the first drop, and provide clear Shop / Story navigation.
2. Shop / collection	Show the Phase 0 products, prices, available sizes, and honest sold-out states. Keep concept mockups separate from products that can actually be purchased.
3. Product details	Product images, description, price, size/variant selection, stock status, add-to-bag action, and clear validation when a required size is missing.
4. Cart / bag	Review selected products, variants, quantities, subtotal, remove/update actions, empty-cart state, and proceed-to-checkout navigation.
5. Checkout review	Review items and delivery information before handing off to a secure hosted checkout or supported payment flow.
6. Order confirmation	Show a real order reference and accurate payment/order status only after the backend or commerce provider confirms it.
7. Story / “What happened?”	Explain the current phase, prior milestones, what Snobi spent money on, and a small tease of what may happen next without revealing the full future plan.
8. Supporters	A verified-only supporter list or leaderboard, with empty states and privacy/opt-out choices. Do not invent supporters or rank mock purchases.
9. FAQ	Answer what a purchase means, what is included, sizing, delivery, limited drops, payment, returns, and how the story works.
10. Contact	Accessible contact form and official social/contact links. Until delivery is configured, show honest feedback that a message was not sent.
11. Shipping, returns & exchanges	Publish only policies that have been decided and can be honored. Do not invent shipping prices, delivery times, or return rights.
12. Privacy policy & terms	Explain data collection, order/contact handling, third-party services, and store terms. These must match the actual implementation and applicable law.
System states	Loading, no products, sold out, validation errors, failed payment, empty cart, success, and not-found/error states; all should be understandable and accessible.

4. Snobi artwork and image direction

Essential initial artwork

- **Primary Phase 0 scene:** a wide 16:9 illustration of Snobi sitting on the ground beside a dusty Egyptian roadside, looking broke, with an empty wallet and no future-phase possessions.
- **Transparent / isolated mascot:** the original Snobi drawing for product cards, small UI placements, or alternate layouts.
- **Product imagery:** clean, consistent images of the actual shirts and their prints. Do not present generated shirt mockups as available inventory unless those exact items will be produced.
- **Story assets:** leave space for future scenes and props; do not generate every future milestone now.

Image-generation prompt used as direction

> *"Create a wide 16:9 crude black-marker drawing of the original Snobi character sitting on the ground beside a dusty Egyptian roadside. Preserve the original mascot's design and proportions exactly. He has almost nothing; include an empty wallet and a sparse, simple roadside setting. Use an off-white paper background and rough hand-drawn lines. No text, no logos, no future-phase objects, no polished 3D look, and no extra characters."*

Important: the prompt is a direction summary, not a guarantee that every generated image will preserve the mascot. Compare generated results against the original drawing and reject versions that alter Snobi's recognizable shape.

5. UI/UX and implementation principles

- Mobile-first and responsive; check both English and Arabic layouts if bilingual support is retained.
- Keep navigation, bag count, product controls, and links genuinely functional—not just visually present.
- Use accessible labels, keyboard focus, readable contrast, clear errors, and meaningful button states.
- Use real product and stock data for purchasable items; mark concept mockups clearly.
- Keep story content separate from store inventory so a new story phase does not require rewriting commerce logic.
- Use real verified data for orders, supporters, and financial progress; distinguish fictional Snobi spending from actual business finances.
- Do not collect payment-card data directly in a custom form. Use a reputable hosted checkout or a properly integrated provider that supports the business and country.

6. Current codebase and where we stopped

Repository and stack

GitHub repository: Seif-shawky/welcome-home

The public repository identifies the project as built with Lovable and includes a README with local development instructions. The project stack observed in the discussion includes React, TypeScript, Vite, TanStack Start / Router, Tailwind CSS, Zustand, React Query, and Vitest. Verify exact versions in package.json before making upgrades.

Current checkpoint

- Lovable was used to create the Phase 0 project and the work was pushed to GitHub.
- The intended page sequence reached Page 12 — Privacy Policy and Terms; the user reported Lovable stopped at that phase.

- The repository roadmap indicates that important real-world integrations are still pending, including real Shopify products, cart/checkout updates, contact delivery, and verified supporter data.
- The roadmap says product mockups should remain non-purchasable until actual products are confirmed.
- Therefore, treat the current project as a frontend-first prototype—not as a confirmed complete backend or live store. Some server capability may exist in the framework; inspect the code to establish what is actually implemented.

Public repository reference checked 28 October 2026: GitHub repository. Repository contents and roadmap can change; this document records the checkpoint, not a live audit.

Run the project locally

```
git clone https://github.com/Seif-shawky/welcome-home.git
```

```
cd welcome-home
```

```
npm install
```

```
npm run dev
```

Open the local URL printed in the terminal. Then run the available quality checks from package.json (for example, npm run build, npm run lint, and npm test if those scripts are present). Fix baseline failures before adding features.

First inspection checklist

- Read package.json, README.md, roadmap.md, and the Vite/TanStack configuration.
- Inspect src/ for routes, reusable components, mock data, and server functions or API endpoints.
- Search for TODO, mock, placeholder, Shopify, checkout, payment, contact, and supporter references.
- Identify which buttons currently only change local UI state and which actions call a real service.
- Check whether environment variables are required; never commit API keys or secrets.
- Run the app and test every route manually on desktop and mobile sizes.
- Record the baseline build/lint/test results in the README or project notes.

7. Choose the commerce architecture

Option A — Custom site + Shopify (recommended for a faster real launch)

- Keep the custom React/TanStack website and visual identity.
- Use Shopify for product catalogue, variants, stock, orders, and hosted checkout, following the integration approach supported by the project.
- Avoid rebuilding inventory, order management, and payment handling unless the project’s learning goals require it.
- Confirm fees, supported payment providers, Egypt-based business requirements, shipping configuration, and any tax/compliance obligations before committing.

Option B — Custom backend (more learning and maintenance)

- Use TanStack Start server functions/routes or a separate backend, plus a database such as PostgreSQL or a managed service such as Supabase.
- Build product/variant/inventory management, orders, order items, payment verification, admin tools, contact delivery, and supporter visibility.

- Use a payment provider that supports the business and country; verify payments server-side and handle provider webhooks safely.
- This route gives more control but requires more code, security work, testing, deployment, and ongoing maintenance.

Decision to make before building further: choose one primary source of truth for products, stock, orders, and payments. Do not build a custom inventory database while also treating Shopify as authoritative unless a deliberate sync design exists.

8. Recommended phased roadmap

Phase 0A — Stabilize the prototype

- Clone and run the project locally.
- Run build, lint, and tests; record failures.
- Map all routes and mark each feature as real, mock, incomplete, or unknown.
- Verify the privacy/terms pages exist and are linked; review the Page 12 stopping point.
- Commit a clean baseline to GitHub.

Phase 0B — Finish the customer-facing UI

- Complete the 12 page areas listed above and all system states.
- Check responsive behavior, bilingual layout if required, accessibility, empty states, and navigation.
- Replace fake success messages with accurate states.
- Finalize product descriptions, sizing, prices, photos, shipping and return policies before publishing them.

Phase 1 — Product catalogue

- Choose Shopify or custom commerce backend.
- Add real products, sizes/variants, prices, stock, product photos, and sold-out behavior.
- Keep concept mockups clearly separate from purchasable inventory.
- Create a minimal admin workflow for updating products and stock.

Phase 2 — Cart and checkout

- Make add-to-bag, quantity changes, variant validation, removal, totals, and bag count work.
- Connect checkout to the chosen commerce/payment provider.
- Verify order and payment status on the server/provider; never trust a client-only success screen.
- Create the confirmation page from the confirmed order record.

Phase 3 — Fulfilment and customer communication

- Configure delivery zones, shipping prices, realistic delivery estimates, returns/exchanges, and order emails.
- Add contact form delivery and test the destination inbox.
- Create operational steps for packing, shipping, cancellations, refunds, and customer support.

Phase 4 — Story and verified supporters

- Create structured phase/story-event data so each new phase can change the scene, text, products, and props.

- Track real milestones carefully; keep fictional character spending separate from real business accounting.
- Add a verified-only supporter feed or leaderboard only after orders and consent/privacy rules work.
- Provide opt-out or private display options; use honest empty states when no verified data exists.

Phase 5 — Launch and observe

- Deploy to a production host, configure domain and HTTPS, and keep secrets in environment variables.
- Test payment sandbox, successful and failed payments, cancellations, refunds, duplicate webhooks, stock changes, and order confirmation.
- Test a complete real order end-to-end before public launch.
- Monitor errors, backups, analytics/privacy settings, and support requests.

Phase 6 — Later story drops

- Release limited drops tied to new story phases.
- Change Snobi's background, props, and circumstances as the story progresses.
- Use Instagram to tease the next phase without revealing all milestones.
- Introduce girlfriend/enemy characters only when they help the story, not just to add complexity.

9. Suggested data model if building custom commerce

Do not implement all of this before the architecture decision. These are candidate entities for a custom backend; Shopify would own much of this data if selected.

Entity	Purpose
products	Product name, description, images, status, collection/phase.
product_variants	Size, SKU, price, stock quantity, relation to product.
orders	Customer/delivery snapshot, order state, currency, totals, provider reference.
order_items	Purchased product/variant snapshot, quantity, price at purchase.
payments	Provider transaction ID, amount, currency, verified status, timestamps.
story_events	Phase, date, narrative, milestone, scene/asset references.
supporter_profiles	Optional public display name, consent/visibility preference, verified order relation.

10. Testing and launch checklist

- A user cannot add a product without selecting a required variant/size.
- Unavailable or sold-out variants cannot be purchased.
- Cart totals are calculated from trusted product data, not values supplied by the browser.
- A payment success page appears only after verified confirmation from the provider/backend.
- Duplicate payment callbacks do not create duplicate orders or double-count milestones.
- Failed/cancelled payments leave the order in an accurate state.

- Inventory is updated safely when orders are completed or cancelled.
- Contact forms validate inputs and genuinely deliver messages—or clearly say they do not yet.
- Supporter names are based on verified orders and the user's display consent.
- Shipping, returns, privacy, and terms match the actual business and service configuration.
- No API keys, payment secrets, or private customer information are committed to GitHub.
- The site works on mobile, keyboard navigation, and the major target browsers.
- At least one end-to-end test order has been completed before launch.

11. Immediate next actions — in order

1. **Clone and run the repository locally.** Confirm the exact start command and that the site loads.
2. **Run the baseline checks.** Build, lint, tests; fix or document pre-existing failures.
3. **Audit the code and roadmap.** List every route and label its functionality: real, mock, incomplete, or unknown.
4. **Decide the commerce approach.** Shopify is the simpler launch path; custom backend is the deeper coding project.
5. **Finish one complete vertical slice.** Real product → size selection → cart → secure checkout → verified order → confirmation.
6. **Only then add the story mechanics.** Connect real milestones and verified supporter data after orders are trustworthy.
7. **Deploy a test version.** Do not announce a public shop until payment, fulfilment, policies, and customer communication have been tested.

The most important next move is not to ask Lovable to add more pages. It is to understand and own the code that already exists, then finish one real end-to-end purchase flow.

12. Open decisions to settle

- Will the store be Shopify-backed or fully custom?
- What exact Phase 0 shirts will actually be manufactured and sold?
- What are the final prices, sizes, stock quantities, and product photographs?
- What shipping areas, costs, delivery estimates, returns, and exchanges can the business honestly support?
- Which payment provider supports the business setup in Egypt?
- Will the site be bilingual from launch, and what is the final Arabic brand voice?
- What data may be shown publicly in the supporter list, and how can a buyer opt out?
- What is the first milestone and what information about real sales or spending will be disclosed?

Appendix — reference links

- GitHub repository — source code, README, and roadmap.
- Lovable — original AI-assisted project builder.
- Local project files to review first: package.json, README.md, roadmap.md, src/, and configuration files.

This document consolidates the decisions, creative direction, page plan, technical assessment, and next steps discussed so far. It is a working plan, not a claim that the listed features are already implemented or that legal/payment policies have been validated.